

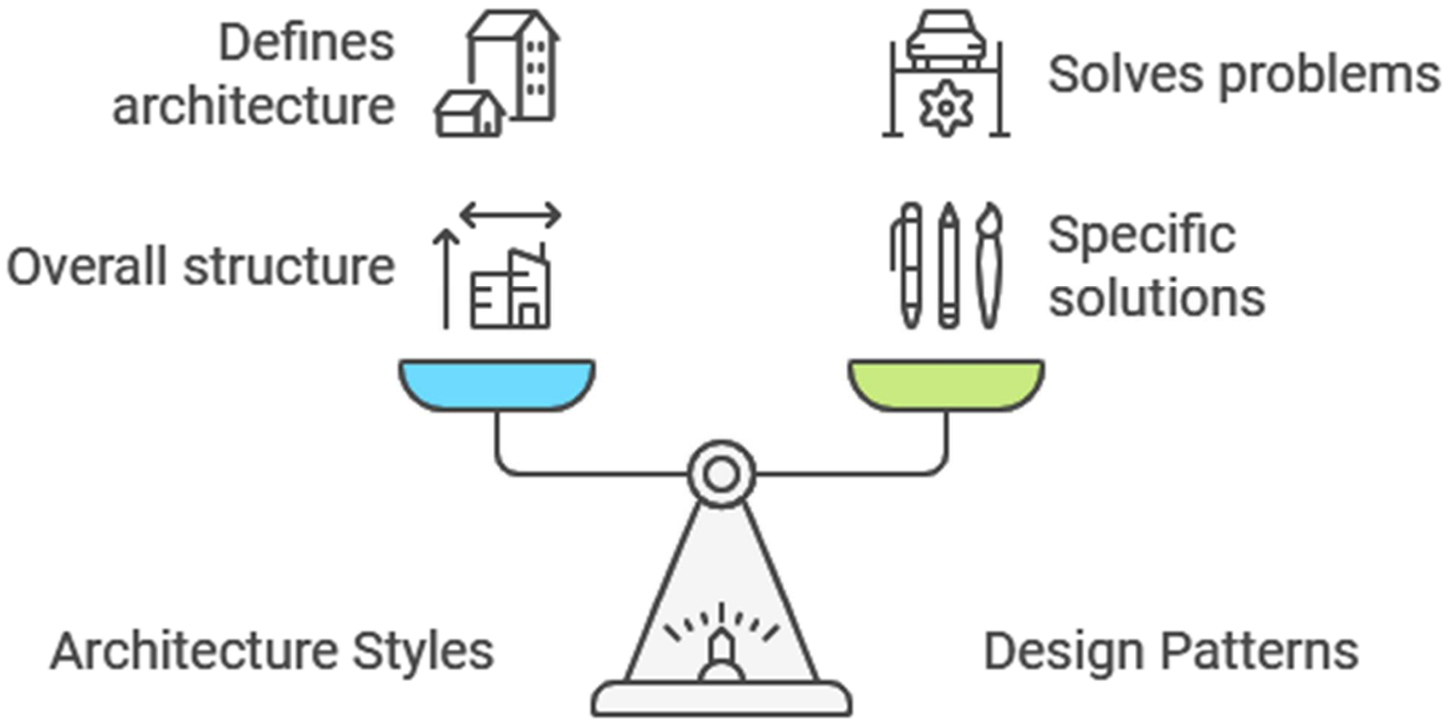
SOFWARE DESIGN & ARCHITECTURE

Chapter 6 - Creational Method in Architecture Design Pattern

Prepared By: Dr Muhammad Khatibsyarbini

Innovating Solutions

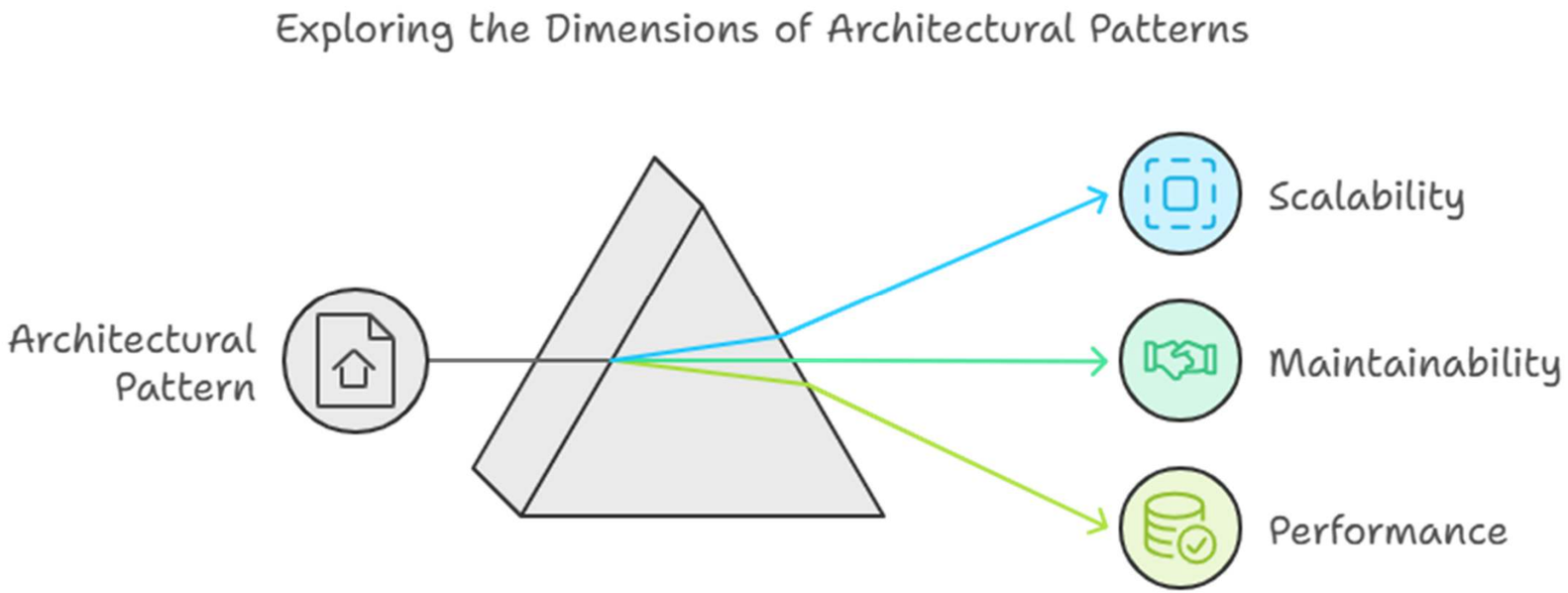
ARCHITECTURAL STYLE vs PATTERN



Styles define structure; patterns provide solutions.

OVERVIEW ARCHITECTURE PATTERN

An **architectural pattern** is a reusable solution to a commonly occurring design problem in software architecture. It serves as a blueprint for how to structure a system, providing guidance on interactions and relationships between components. Patterns complement architectural styles by focusing on solutions within specific contexts, such as scalability, maintainability, or performance.



Creational Design Patterns are used to manage and simplify the creation of objects in software architecture, focusing on flexibility, scalability, and reusability. These patterns help decouple the object creation process from the system, ensuring that the code is easier to maintain and extend.

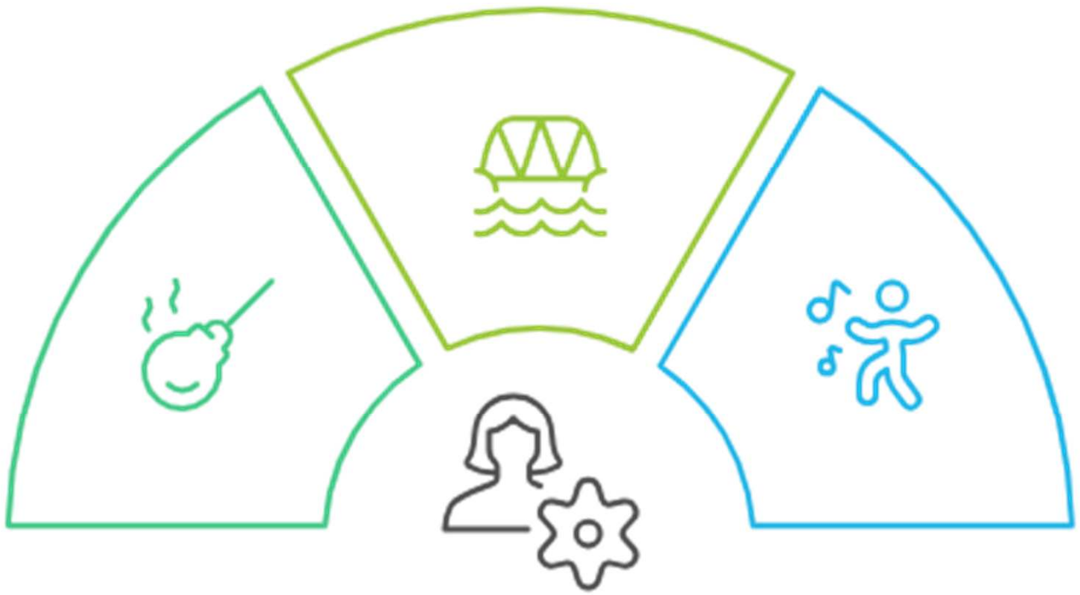
Design Patterns

Structural Patterns

Patterns that define object composition and relationships.

Creational Patterns

Patterns focused on object creation mechanisms.



Behavioral Patterns

Patterns that manage object interaction and responsibility.

Enhancing Software Architecture with Creational Patterns

Decoupled Creation

Separates object creation from system logic

Flexibility

Allows adaptation to changing requirements and environments



Reusability

Enables using existing components in new contexts

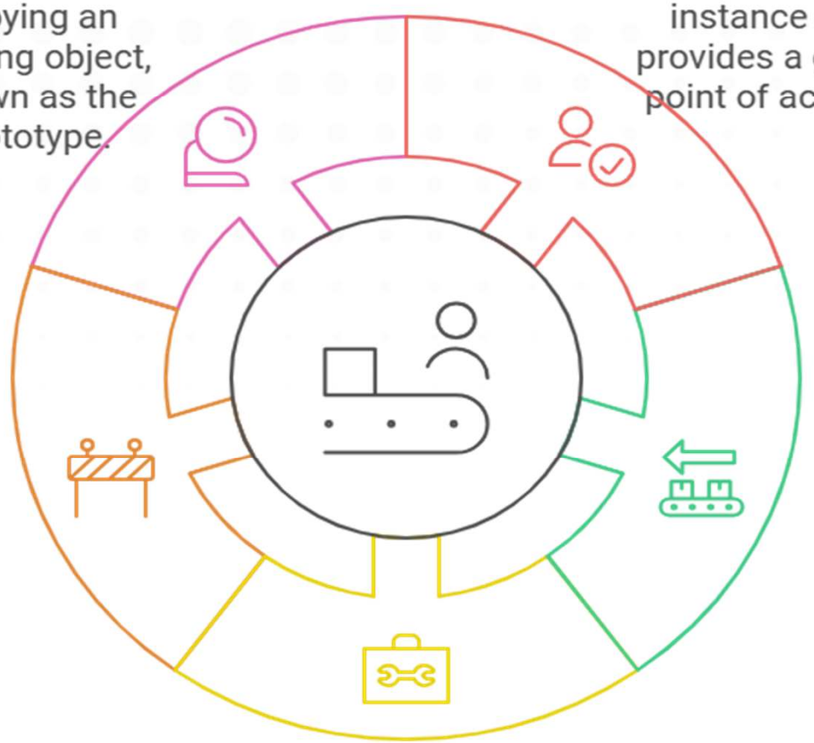
Scalability

Supports growth and expansion without major changes

Creational Design Patterns

Creational Patterns

- **Definition**: Concerned with object creation mechanisms, aiming to make the system independent of how objects are instantiated.
- **Focus**: Deals with class instantiation and object composition, ensuring flexibility in creating objects.
- **Key Idea**: Provides flexibility in the process of creating objects, ensuring proper initialization and object reuse.



Prototype

Creates new objects by copying an existing object, known as the prototype.

Singleton

Ensures a class has only one instance and provides a global point of access.

Builder

Separates the construction of a complex object from its representation, allowing the same construction process to create different representations.

Factory Method

Defines an interface for creating objects, allowing subclasses to alter the type of objects created.

Abstract Factory

Provides an interface for creating families of related or dependent objects without specifying their concrete classes.

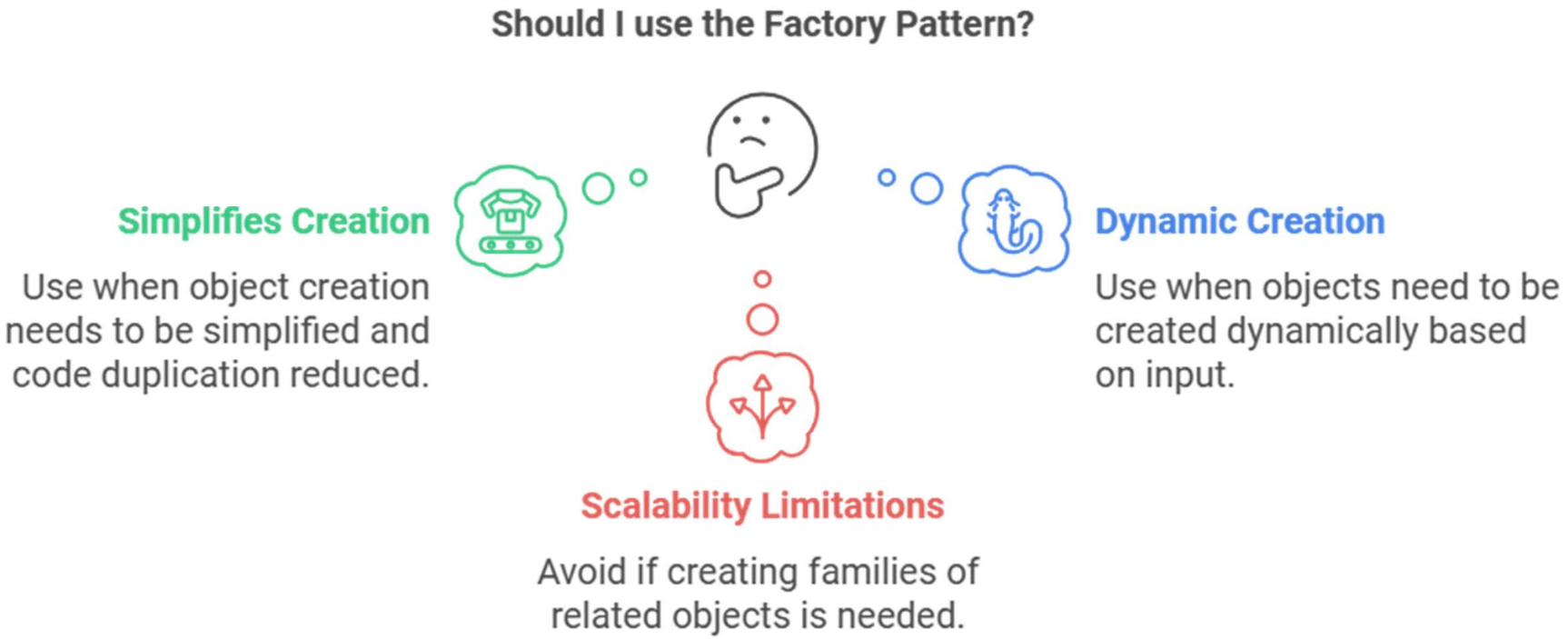
Comparison of Creational Patterns

Pattern	Key Purpose	Example Use Case
Singleton	Single instance for global access.	Configuration manager or logging system.
Factory	Creates objects dynamically.	User controllers based on roles.
Abstract Factory	Creates families of related objects.	Product catalog and their UI views.
Builder	Builds complex objects incrementally.	Order creation with configurable options.
Prototype	Clones existing objects.	User preferences or UI themes.

www.utm.my

Factory Pattern

- **Purpose** : Creates objects without exposing the instantiation logic.
- **When to Use** : To create objects dynamically based on input (e.g., user roles).
- **Advantages** : Simplifies object creation and reduces code duplication.
- **Disadvantages** : Limited scalability for creating families of related objects.



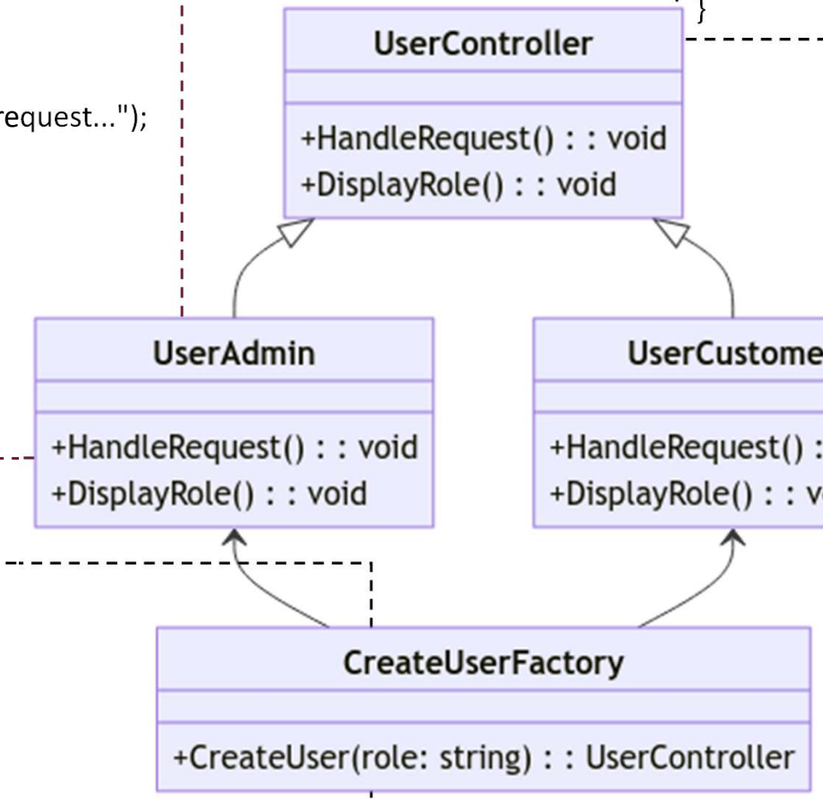
www.utm.my

Factory Patterns

```
public abstract class UserController
{
    public abstract void
HandleRequest();
    public abstract void DisplayRole();
}
```

```
public class UserAdmin : UserController
{
    public override void HandleRequest()
    {
        Console.WriteLine("Handling admin-specific request...");
    }

    public override void DisplayRole()
    {
        Console.WriteLine("Role: Admin");
    }
}
```




```
public class UserCustomer : UserController
{
    public override void HandleRequest()
    {
        Console.WriteLine("Handling customer-specific request...");
    }

    public override void DisplayRole()
    {
        Console.WriteLine("Role: Customer");
    }
}
```

```
public class CreateUserFactory
{
    public UserController CreateUser(string role)
    {
        return role switch
        {
            "Admin" => new UserAdmin(),
            "Customer" => new UserCustomer(),
            _ => throw new ArgumentException("Invalid role specified")
        };
    }
}
```

Factory Pattern

Usage Example

```
using System;

class Program
{
    static void Main(string[] args)
    {
        CreateUserFactory userFactory = new CreateUserFactory();

        // Create an Admin User
        UserController adminUser = userFactory.CreateUser("Admin");
        adminUser.DisplayRole();        // Output: Role: Admin
        adminUser.HandleRequest();      // Output: Handling admin-specific request...

        Console.WriteLine();

        // Create a Customer User
        UserController customerUser = userFactory.CreateUser("Customer");
        customerUser.DisplayRole();     // Output: Role: Customer
        customerUser.HandleRequest();   // Output: Handling customer-specific request...
    }
}
```

Output:

```
Role: Admin
Handling admin-specific request...

Role: Customer
Handling customer-specific request...
```

- **Factory Class: CreateUserFactory**
  - Encapsulates the logic of creating different types of UserController objects based on a given role (e.g., "Admin" or "Customer").
- **Product Interface: UserController**
  - Acts as a common interface for all user types (e.g., Admin and Customer).
- **Methods**:
  - HandleRequest(): Represents the actions specific to the user type.
  - DisplayRole(): Outputs the role of the user.
- **Concrete Products: UserAdmin and UserCustomer**
  - These are specific implementations of the UserController interface:
  - **UserAdmin**: Implements logic specific to an admin user.
  - **UserCustomer**: Implements logic specific to a customer user.
- **Client Code (Main Method)**:
  - The client code uses the factory to create user objects (UserAdmin or UserCustomer) dynamically, based on the input role.

www.utm.my

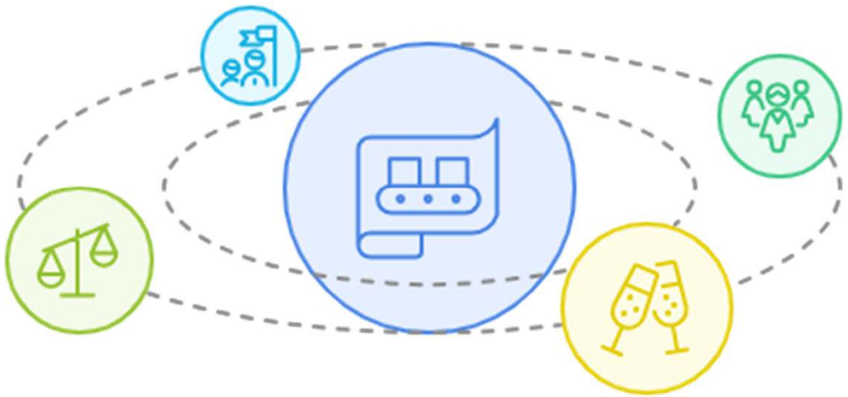
Abstract Factory Pattern

Purpose

Creates families of related objects

When to Use

When a system needs grouped objects



Advantages

Ensures consistency and easy swapping

Disadvantages

More complex with multiple classes

Should the Abstract Factory Pattern be used?

Use for Consistency

Ensures consistency across product families and allows easy swapping.

Avoid Complexity

Avoids the complexity of managing multiple classes and interfaces.

www.utm.my

Abstract Factory Pattern

```
public class MaleFashionFactory :
FashionFactory
{
    public override Shirt CreateShirt()
    {
        return new MaleShirt();
    }

    public override Pant CreatePant()
    {
        return new MalePant();
    }
}
```

```
public abstract class FashionFactory
{
    public abstract Shirt CreateShirt();
    public abstract Pant CreatePant();
}
```

```
public class FemaleFashionFactory :
FashionFactory
{
    public override Shirt CreateShirt()
    {
        return new FemaleShirt();
    }

    public override Pant CreatePant()
    {
        return new FemalePant();
    }
}
```

```
public class MaleShirt : Shirt
{
    public override void DisplayInfo()
    {
        Console.WriteLine("This is a male shirt.");
    }
}

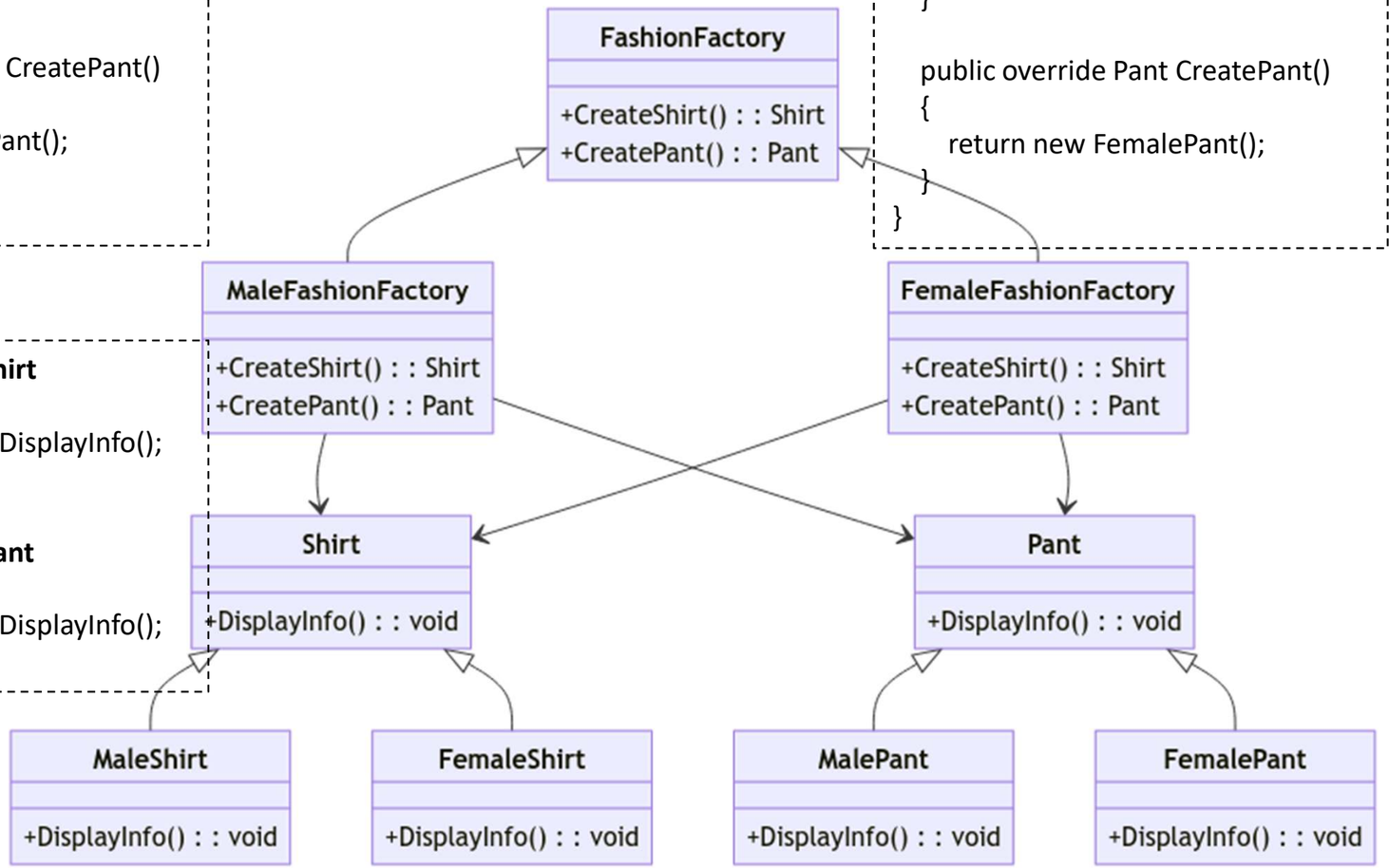
public class FemaleShirt : Shirt
{
    public override void DisplayInfo()
    {
        Console.WriteLine("This is a female shirt.");
    }
}

public class MalePant : Pant
{
    public override void DisplayInfo()
    {
        Console.WriteLine("This is a male pant.");
    }
}

public class FemalePant : Pant
{
    public override void DisplayInfo()
    {
        Console.WriteLine("This is a female pant.");
    }
}
```

```
public abstract class Shirt
{
    public abstract void DisplayInfo();
}

public abstract class Pant
{
    public abstract void DisplayInfo();
}
```



www.utm.my

Abstract Factory Pattern

Usage Example

```
using System;

class Program
{
    static void Main(string[] args)
    {
        // Create a Male Fashion Factory
        FashionFactory maleFactory = new MaleFashionFactory();

        // Create male fashion products
        Shirt maleShirt = maleFactory.CreateShirt();
        Pant malePant = maleFactory.CreatePant();

        // Display male fashion details
        maleShirt.DisplayInfo();  // Output: This is a male shirt.
        malePant.DisplayInfo();   // Output: This is a male pant.

        Console.WriteLine();

        // Create a Female Fashion Factory
        FashionFactory femaleFactory = new FemaleFashionFactory();

        // Create female fashion products
        Shirt femaleShirt = femaleFactory.CreateShirt();
        Pant femalePant = femaleFactory.CreatePant();

        // Display female fashion details
        femaleShirt.DisplayInfo();  // Output: This is a female shirt.
        femalePant.DisplayInfo();   // Output: This is a female pant.
    }
}
```

Output

When you run the program, the output will be:

```
This is a male shirt.
This is a male pant.

This is a female shirt.
This is a female pant.
```

Explanation

1.Abstract Classes:
- Shirt and Pant define the common interface for all shirt and pant types.

2.Concrete Classes:
- MaleShirt, FemaleShirt, MalePant, and FemalePant implement their respective DisplayInfo methods.

3.Abstract Factory:
- FashionFactory defines the creation methods for Shirt and Pant.

4.Concrete Factories:
- MaleFashionFactory and FemaleFashionFactory provide specific implementations for creating male and female fashion items.

5.Usage:
- The client code creates factories (MaleFashionFactory or FemaleFashionFactory) and uses them to generate appropriate products (Shirt and Pant) without worrying about their concrete classes.

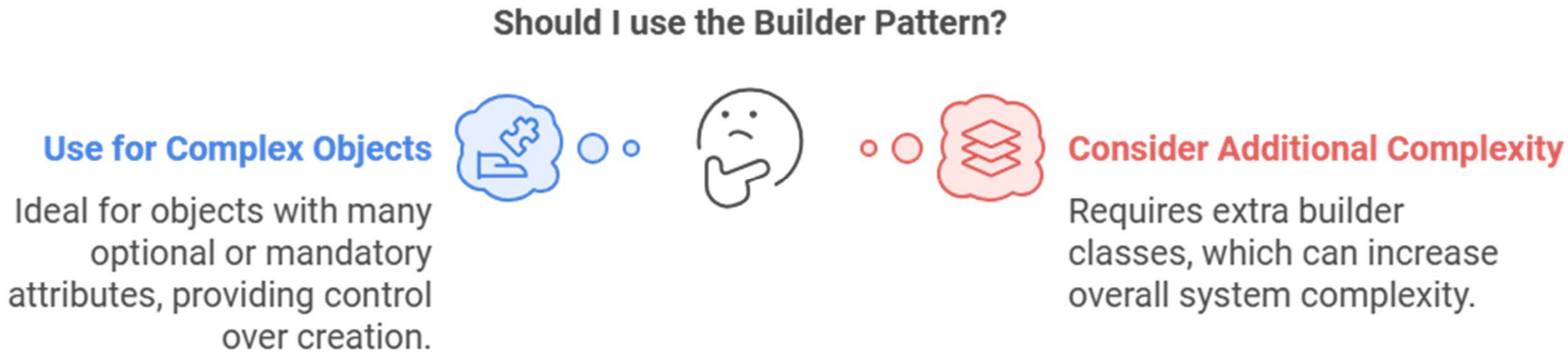
Factory vs Abstract Factory

Aspect	Factory Pattern	Abstract Factory Pattern
Purpose	Create a single product or type of object.	Create families of related or dependent objects.
Number of Products	Focuses on one product or closely related types.	Focuses on creating a family of related products.
Complexity	Simpler to implement.	More complex, with multiple interfaces/classes.
Example	Create UserController (AdminController, CustomerController).	Create ElectronicsProduct, FashionProduct, and their views.
Scalability	Limited scalability for related products.	Scalable for families of related products.
Dependency	Relies on subclassing.	Relies on interfaces and multiple factories.
Flexibility	Changes require modifying the factory method.	Families can be swapped without changing client code.

www.utm.my

Builder Pattern

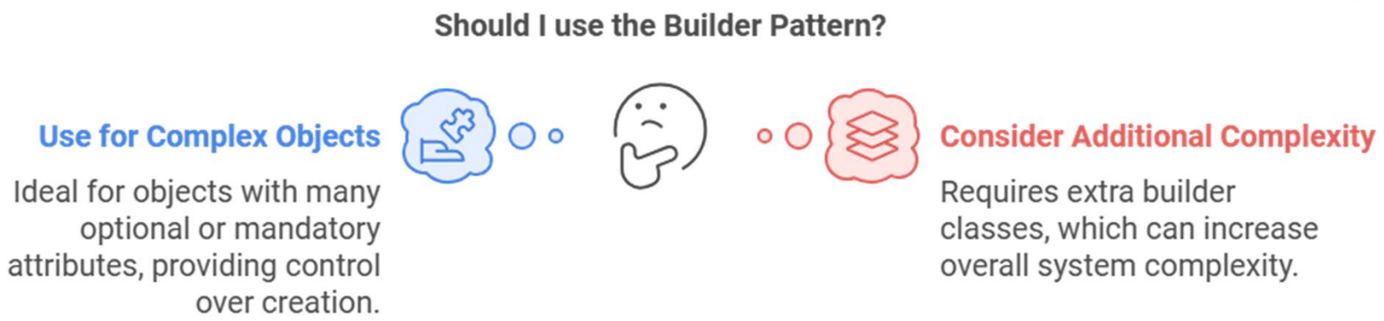
- **Purpose** : Constructs complex objects step by step with varying configurations.
- **When to Use** : For objects with many optional or mandatory attributes (e.g., creating orders or reports).
- **Advantages** : Provides control over the object creation process.
- **Disadvantages** : Requires additional builder classes, increasing complexity.

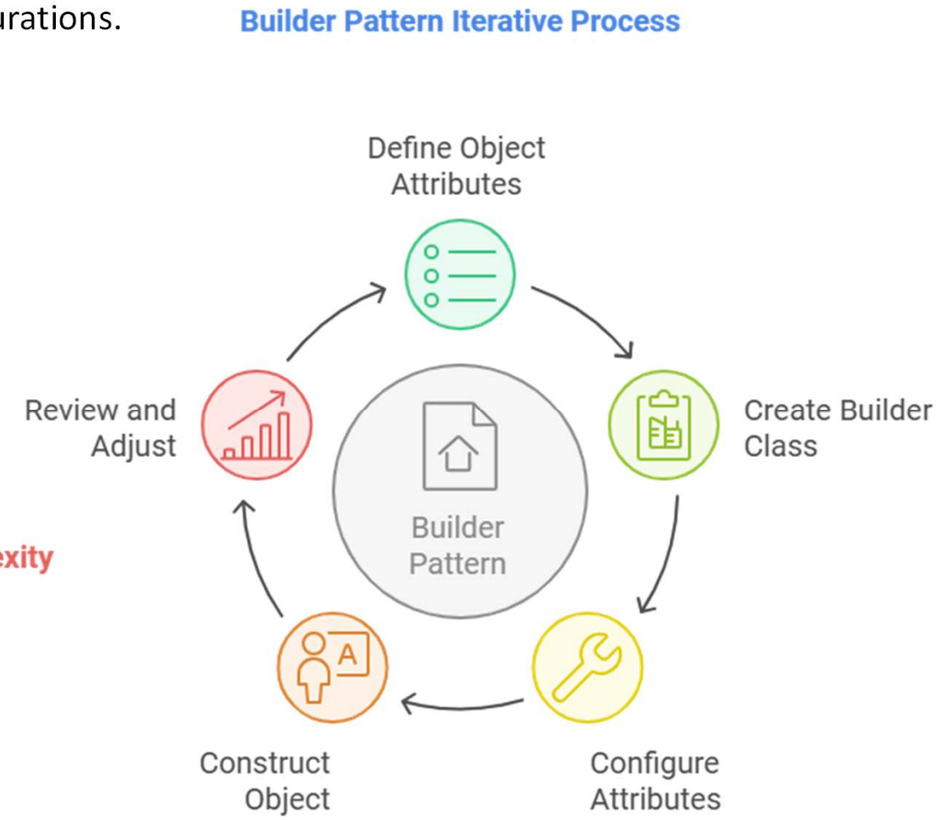


www.utm.my

Builder Pattern

- **Purpose** : Constructs complex objects step by step with varying configurations.
- **When to Use** : For objects with many optional or mandatory attributes
- **Advantages** : Provides control over the object creation process.
- **Disadvantages** : Requires additional builder classes, increasing complexity.





www.utm.my

Builder Pattern

```
using System;
using System.Collections.Generic;

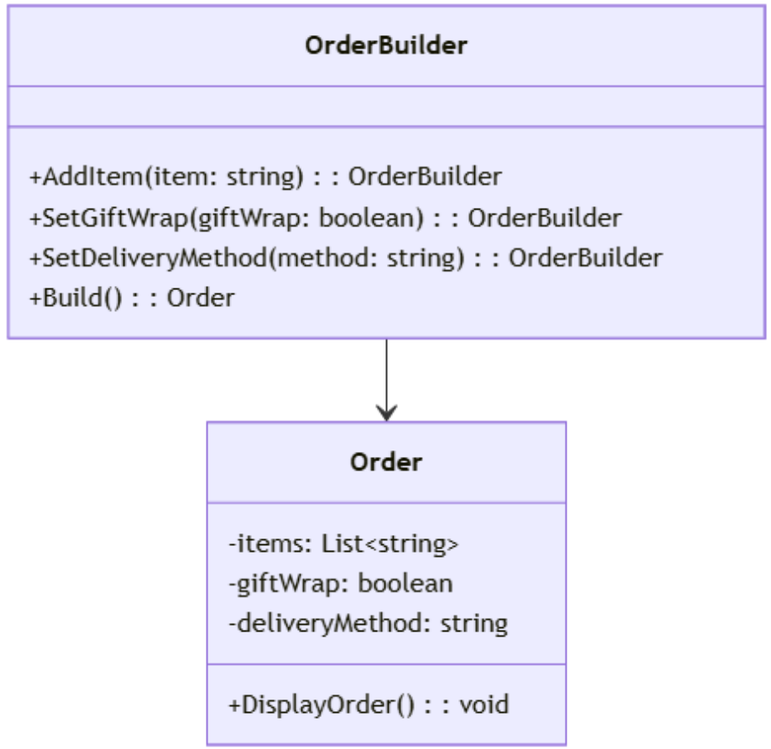
public class Order
{
    private List<string> items = new List<string>();
    private bool giftWrap;
    private string deliveryMethod;

    public void SetItems(List<string> items)
    {
        this.items = items;
    }

    public void SetGiftWrap(bool giftWrap)
    {
        this.giftWrap = giftWrap;
    }

    public void SetDeliveryMethod(string method)
    {
        this.deliveryMethod = method;
    }

    public void DisplayOrder()
    {
        Console.WriteLine("Order Summary:");
        Console.WriteLine("Items: " + string.Join(", ", items));
        Console.WriteLine("Gift Wrap: " + (giftWrap ? "Yes" : "No"));
        Console.WriteLine("Delivery Method: " + deliveryMethod);
    }
}
```




```
public class OrderBuilder
{
    private List<string> items = new List<string>();
    private bool giftWrap = false;
    private string deliveryMethod = "Standard";

    public OrderBuilder AddItem(string item)
    {
        items.Add(item);
        return this; // Enables method chaining
    }

    public OrderBuilder SetGiftWrap(bool giftWrap)
    {
        this.giftWrap = giftWrap;
        return this; // Enables method chaining
    }

    public OrderBuilder SetDeliveryMethod(string method)
    {
        this.deliveryMethod = method;
        return this; // Enables method chaining
    }

    public Order Build()
    {
        Order order = new Order();
        order.SetItems(items);
        order.SetGiftWrap(giftWrap);
        order.SetDeliveryMethod(deliveryMethod);
        return order;
    }
}
```

www.utm.my

Builder Pattern

```
class Program
{
    static void Main(string[] args)
    {
        // Using the builder to create an order
        OrderBuilder builder = new OrderBuilder();

        Order myOrder = builder
            .AddItem("Laptop")
            .AddItem("Mouse")
            .SetGiftWrap(true)
            .SetDeliveryMethod("Express")
            .Build();

        myOrder.DisplayOrder();

        Console.WriteLine();

        // Another order with default gift wrap and delivery
        Order anotherOrder = builder
            .AddItem("Book")
            .SetGiftWrap(false)
            .Build();

        anotherOrder.DisplayOrder();
    }
}
```

Output

```
Order Summary:
Items: Laptop, Mouse
Gift Wrap: Yes
Delivery Method: Express

Order Summary:
Items: Book
Gift Wrap: No
Delivery Method: Express
```

Explanation of the Builder Pattern

The Builder Pattern is a creational design pattern that allows the construction of complex objects step-by-step. It separates the construction process from the representation of the object, making it easier to create objects with varying configurations.

How It Works in This Example

1.Order Class:

- Represents the complex object being built.
- Contains properties like items, giftWrap, and deliveryMethod that describe an order.
- Provides a method DisplayOrder() to display the order details.

2.OrderBuilder Class:

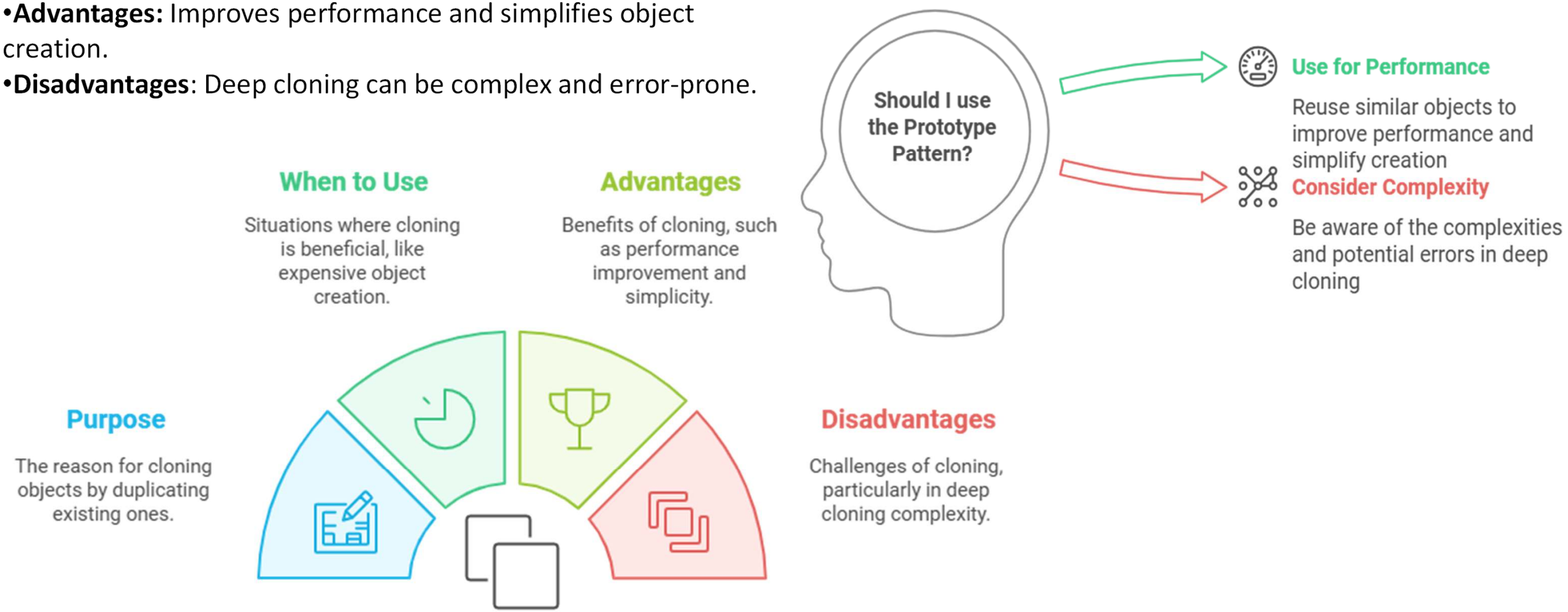
- Provides step-by-step methods to configure the Order object:
  - AddItem(string item) adds an item to the order.
  - SetGiftWrap(bool giftWrap) sets whether the order should be gift-wrapped.
  - SetDeliveryMethod(string method) specifies the delivery method.
- Implements the **method chaining pattern**, allowing the client to configure the order fluently.
- The Build() method assembles the final Order object with the accumulated configurations.

3.Client Code:

- The client code (Main method) uses the builder to construct the Order object step-by-step.
- The builder provides flexibility in constructing objects with varying configurations without the need to directly manipulate the Order object.

Prototype Pattern

- **Purpose**: Creates new objects by cloning existing ones.
- **When to Use**: When object creation is expensive, and you want to reuse similar objects (e.g., user preferences).
- **Advantages:** Improves performance and simplifies object creation.
- **Disadvantages**: Deep cloning can be complex and error-prone.



www.utm.my

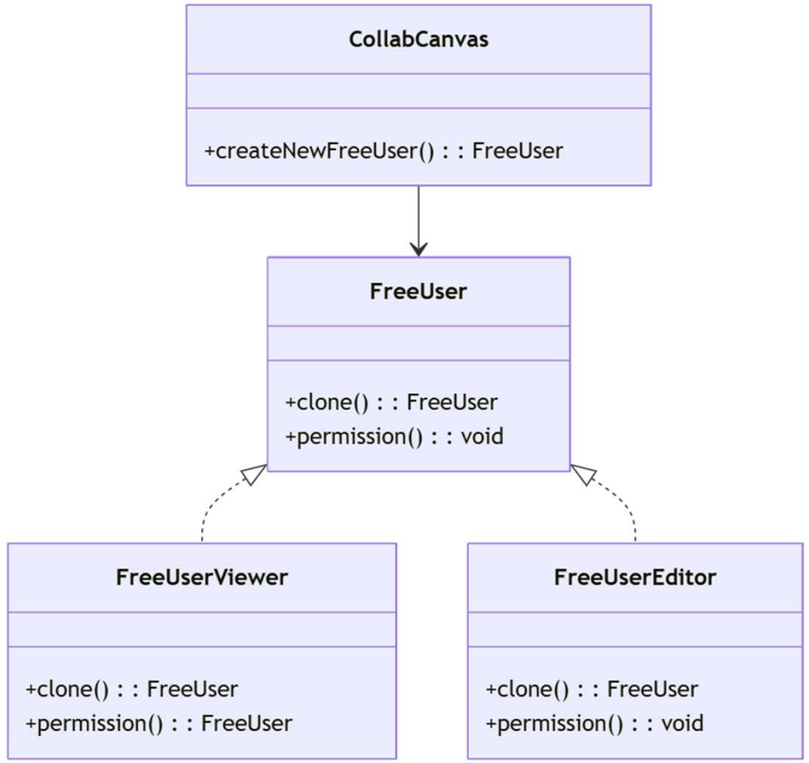
Prototype Pattern

Prototype: User Preferences/Cloning Users

- **Feature**: Clone user preferences for onboarding similar users.
- **Reason**: Simplify object creation by copying existing configurations.

```
public abstract class FreeUser
{
    // Clone method to be implemented by subclasses
    public abstract FreeUser Clone();

    // Abstract method to specify permissions
    public abstract void Permission();
}
```




```
public class CollabCanvas
{
    private FreeUser prototype;

    // Constructor to initialize the prototype
    public CollabCanvas(FreeUser prototype)
    {
        this.prototype = prototype;
    }

    // Create a new free user by cloning the prototype
    public FreeUser CreateNewFreeUser()
    {
        return prototype.Clone();
    }
}
```

```
public class FreeUserViewer : FreeUser
{
    public override FreeUser Clone()
    {
        // Return a new instance of FreeUserViewer
        return new FreeUserViewer();
    }

    public override void Permission()
    {
        Console.WriteLine("Permission: Viewer - Can only view content.");
    }
}
```

```
public class FreeUserEditor : FreeUser
{
    public override FreeUser Clone()
    {
        // Return a new instance of FreeUserEditor
        return new FreeUserEditor();
    }

    public override void Permission()
    {
        Console.WriteLine("Permission: Editor - Can view and edit content.");
    }
}
```

www.utm.my

Prototype Pattern

```
using System;

class Program
{
    static void Main(string[] args)
    {
        // Create a prototype for FreeUserViewer
        FreeUser viewerPrototype = new FreeUserViewer();

        // Initialize CollabCanvas with Viewer prototype
        CollabCanvas canvas = new CollabCanvas(viewerPrototype);

        // Create a new Viewer user
        FreeUser viewerUser = canvas.CreateNewFreeUser();
        viewerUser.Permission(); // Output: Permission: Viewer - Can only view content.

        // Switch prototype to FreeUserEditor
        FreeUser editorPrototype = new FreeUserEditor();
        canvas = new CollabCanvas(editorPrototype);

        // Create a new Editor user
        FreeUser editorUser = canvas.CreateNewFreeUser();
        editorUser.Permission(); // Output: Permission: Editor - Can view and edit content
    }
}
```

```
Permission: Viewer - Can only view content.
Permission: Editor - Can view and edit content.
```

How It Works

1.Prototype Pattern:

- The FreeUser class defines the Clone() method as an abstract method. Each subclass (FreeUserViewer and FreeUserEditor) implements this method to return a copy of itself.
- The CollabCanvas class uses the prototype to create new free users dynamically by calling the Clone() method.

2.Dynamic Switching:

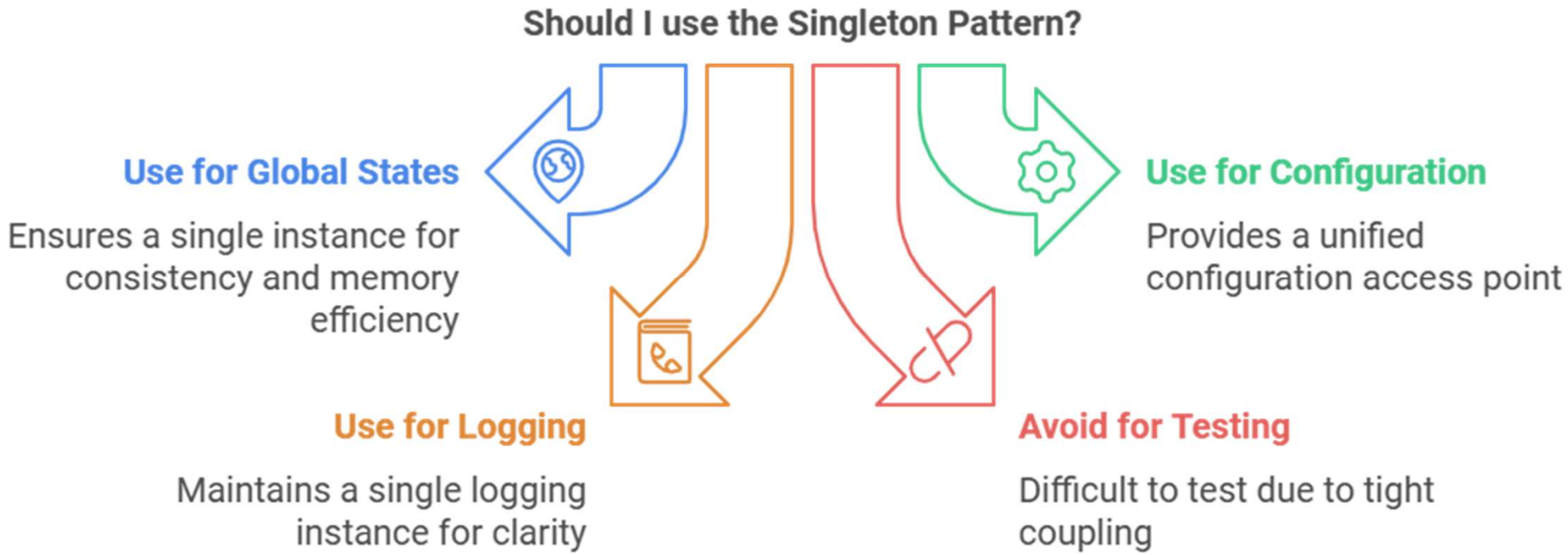
- By switching the prototype in CollabCanvas, you can dynamically create different types of users (e.g., viewers, editors).

3.Permissions:

- The Permission() method is overridden in each subclass to define the specific permissions for that type of user.

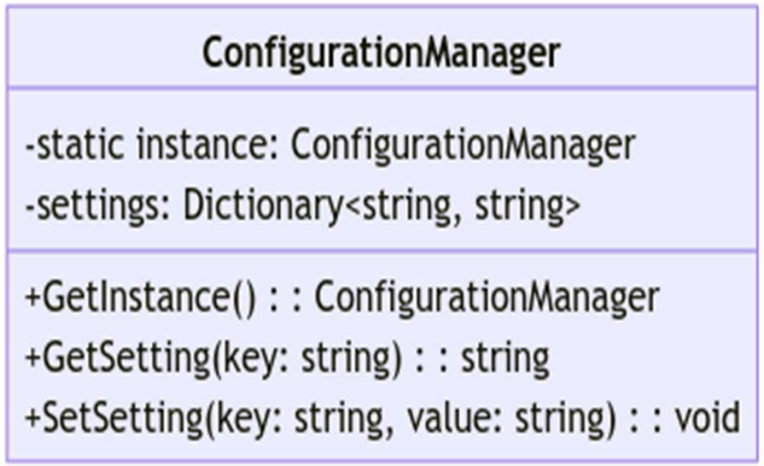
Singleton Pattern

- **Purpose** : Ensures a class has only one instance and provides a global access point.
- **When to Use** : For managing global states, configuration settings, or logging systems.
- **Advantages** : Saves memory and ensures consistency.
- **Disadvantages** : Hard to test and may introduce tight coupling.



www.utm.my

Singleton Pattern




```
public class ConfigurationManager
{
   private static ConfigurationManager _instance;
   private static readonly object _lock = new object();
   private Dictionary<string, string> _settings = new();

   private ConfigurationManager() { }

   public static ConfigurationManager GetInstance()
   {
      lock (_lock)
      {
         if (_instance == null)
         {
            _instance = new ConfigurationManager();
         }
      }
      return _instance;
   }

   public string GetSetting(string key) => _settings.ContainsKey(key) ? _settings[key] : null;

   public void SetSetting(string key, string value) => _settings[key] = value;
   }
```

www.utm.my

Comparison Patterns

Pattern	Key Purpose	Example Use Case
Singleton	Single instance for global access.	Configuration manager or logging system.
Factory	Creates objects dynamically.	User controllers based on roles.
Abstract Factory	Creates families of related objects.	Product catalog and their UI views.
Builder	Builds complex objects incrementally.	Order creation with configurable options.
Prototype	Clones existing objects.	User preferences or UI themes.

www.utm.my

Summary

Pattern	Description	Advantages	Location in MVC	Application in Microservices
Singleton	Ensures a class has only one instance and provides a global point of access to it.	- Controlled access to instance - Lazy initialization - Reduced memory overhead	**Controller or Model**: Manages global objects like configurations, database connections, or session managers.	Used to manage centralized components like **service registry**, **logging**, or **configuration management**.
Factory Method	Defines an interface for creating objects but allows subclasses to alter the type of objects that will be created.	- Simplifies object creation - Promotes loose coupling	**Controller**: Decides which service class to instantiate based on user input or route parameters.	Used in services to decide which implementation of an interface (e.g., database client or message broker) to instantiate dynamically.
Abstract Factory	Provides an interface for creating families of related or dependent objects without specifying their concrete classes.	- Ensures consistency among related objects - Promotes scalability	**Model**: Manages families of related entities (e.g., creating different types of database connections).	Used to create families of services or components that work together, such as factories for **API clients** or **middleware**.
Builder	Separates the construction of a complex object from its representation, allowing the same construction process to create different results.	- Handles complex object creation - Improves readability - Encourages immutability	**Model**: Constructs complex objects like responses, database queries, or view models.	Used for creating complex configurations or objects (e.g., building configuration objects for APIs, pipelines, or deployment manifests).
Prototype	Creates new objects by copying an existing object, instead of creating from scratch.	- Avoids resource-heavy object creation - Efficient cloning - Simplifies object initialization	**Model**: Clones objects like user sessions, cached objects, or templates for repetitive data creation.	Used to replicate stateful service instances or cached objects efficiently, such as **replicating request templates** or session state in distributed systems.

THANK YOU

univteknologimalaysia utm.my utmofficial



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Innovating Solutions

Innovating Solutions

Menginovasi Penyelesaian

Menginovasi Penyelesaian